

ABSTRACT

Aircraft engines are extremely important components that contribute to the increased flight safety and flight efficiency. But in case the invisible breakdowns of such engines take place, then it could lead to the safety factors, inefficiency in flights and high maintenance expenses. The traditional ways of maintenance management of reactive and preventive maintenance are not necessarily effective to detect possible signs of engine degradation and might cause non-productive maintenance. This particular paper is a predictive maintenance model of aircraft engines designed using AI to overcome limitations of digital twin technology concerning engine maintenance prediction according to the dataset of NASA C-MAPSS FD001. In order to predict the RUL as well as the engine's state throughout the start and stop, the proposed method makes use of multivariate sensor variables, including temperature, pressure, speed, and other pertinent aircraft engine characteristics. Such data has been processed using a number of preprocessing techniques, including time sequence creation, RUL data labelling, sensor selection, and sensor data normalisation. Some neural network designs have been repeated, and the final design will be a hybrid Transformer-GRU, which will train both short-term dynamics and long-term degradation patterns. An online twin will be formed that would provide the virtual impression regarding the engine degradation that can be utilized to perform the continuous wellbeing upkeep and anticipatory upkeep studies. The predictive power of the model is very good and the validation R^2 and RMSE of the model are around 0.91 and 12.26 respectively and test R^2 and RMSE of the model are around 0.89 and 13.48 respectively. These findings confirm the validity of the design to generate forecasts about the RUL and are able to be implemented to facilitate risk-averse and less expensive, more evidence-based aircraft engine maintenance planning.

CHAPTER 1

INTRODUCTION

1.1 Overview

In the modern turbofan engines there are a large number of onboard sensors and scans the thermodynamic and mechanical position of the working engines and transmits useful information regarding the condition of the engines and wear out. In this work, the special attention is paid to the following temperature sensors: T2 - temperature at the entrance of the fan, T24 - temperature at the exit of the low-pressure compressor, T30 - temperature at the exit of the high-pressure compressor, and T50 - temperature at the exit of the low-pressure turbine, temperature sensors P2 and P15, P30 and the speed of the fan Nf. The multivariate sensor values are the time series paths that may as well be described as indicators of slow progressive change of the engine state into failed or malfunctioning mode.

A typical prognostics of turbofan engine would most likely rely on the NASA C-MAPSS FD001 dataset. This collection of data is represented as artificial and simulated run-to-failure data of an array of engines and favourable conditions and the labels of Remaining Useful Life. The information set might be directly transferred to the design and experimentation of the resource consuming processes of using the historical sensor data in the forecast of the quantity of productive engine operating cycles before the need to carry out a replacement, and a maintenance practice. Estimation of Remaining Useful Life is highly important in the decision making, most importantly reduction of unexpected failure incidences and proper arrangement of the utilization of the expensive engine parts.

Under the framework of studies, the current paper will center its attention on an AI predictive model of predictive maintenance of turbofan engines, under which multivariate data of the NASA C-MAPSS FD001 dataset would be modeled and simulated to estimate the process of an engine with turbofan and predictive Remaining Useful Life of an engine with the assistance of digital twins. It is estimated to optimize the processes of airline transport as it would make them predictable and understandable.

1.2 Problem Statement

However, as much as considerable progress has been achieved in the instrumentation of aircraft engines, the quantity of relevant maintenance operations that could utilize the enormous volume of data that is present that can be utilised in evaluating engine health is relatively large. Most of the decisions taken in regard to maintenance are usually founded on pre-defined maintenance plans, predefined settings and trend-based analysis that are not adequate in reflecting engine degradation that may have been revealed to varying working environments. There is also a possibility of untimely maintenance measures which could lead to higher operation expenses with these methods, and they can also jeopardize the flight safety.

Among them is the largest which is connected with the accurate determination of the useful life that remains and loss of multivariate sensor information in determining the early life maintenance of the aircraft engine. The above-stated peculiarity, non-reliability over time, and deviation under various working circumstances of the typical and conventional machine learning techniques could very well be cast as the factors behind the ineffectiveness of the methods in handling the nonlinear relationships in engine health assessment, which might lead to a devastating consequence in the results that are pertaining to the engine health assessment.

The existing methods of RUL prediction do not possess the same degree of interpretability as such that it is hard to count on this aspect to apply to applications involving safety. Then, there is the need of a plausible, interpretable AI-based model founded on multivariate time-series data based on the study of the health of turbofan engines. Maintenance-related information about decision making can be replicated with digital twin predictive maintenance capabilities because of their ability to replicate the intricate behaviors.

1.3 Objectives

- Development of a digital copy of aircraft engines for real-time performance and health monitoring to assist condition-based maintenance.
- To propose an AI-based predictive maintenance model for estimating RUL and thus predicting possible failures in turbofan engines by the significant use of multivariate sensor data.
- Development of an intuitive dashboard using this data to display engine health, predictive insights, and maintenance alerts in order to assist with making decisions processes of maintenance.

CHAPTER 2

LITERATRE SURVEY

[1] Deepika J. et al. (2025) – Predictive Maintenance of Aircraft Engines: Machine Learning Approaches for Remaining Useful Life Estimation. Uses NASA C-MAPSS FD001 and traditional ML (XGBoost, RF, SVM, etc.) for RUL prediction, with PostgreSQL and Flask for data handling and visualization. XGBoost performs best ($RMSE \approx 23.8$, $R^2 \approx 0.67$), and XAI feature-importance is used to interpret sensor contributions, showing ML as a transparent but still limited baseline compared to stronger temporal models.

[2] An Integrated XGBoost and BiLSTM Model for Advanced Remaining Useful Life (RUL) Predictions in Aircraft Maintenance. Proposes a hybrid framework where BiLSTM extracts temporal degradation features from engine sensor sequences and XGBoost refines the final RUL estimate. The integrated model outperforms standalone BiLSTM or XGBoost in accuracy and robustness, highlighting the value of combining deep temporal encoders with gradient-boosting for prognostics.

[3] Advanced Remaining Useful Life (RUL) Predictions in Aircraft Engines (FusionRUL-Net, 2025). Introduces FusionRUL-Net, a hybrid deep model combining multi-scale 1D CNNs and Transformer encoders with a gated fusion module for RUL prediction on C-MAPSS. The model achieves very low RMSE, outperforming baselines such as LSTM, BiLSTM-Attention, CNN-LSTM, XGBoost, and XGBoost-BiLSTM, demonstrating the strength of CNN+Transformer hybrids for multivariate engine data.

[4] Interpretable Ensemble RUL Prediction Enables Dynamic Maintenance Scheduling for Aircraft Engines (2025). Presents an ensemble RUL framework that combines multiple prognostic models and couples the predictions with a cost-aware maintenance scheduling strategy. The work emphasizes interpretability of ensemble outputs and shows that accurate RUL plus optimized scheduling can drastically lower the cost of maintenance and risk compared with periodic strategies.

[5] Predictive Maintenance Programs for Aircraft Engines Based on Remaining Useful Life Prediction (2025). Proposes a data-driven predictive-maintenance framework where an RUL model (integrated Transformer–LSTM) is used to trigger dynamic survival-based maintenance strategies. Using C-MAPSS as a case study, the method improves safety, reduces unexpected failures, and optimizes maintenance cost and engine availability versus traditional time-based maintenance.

[6] A Deep Learning-Based Prognostic Approach for Predicting Turbofan Engine Degradation and RUL (CAELSTM, 2025). Develops a Convolutional Autoencoder + Attention-LSTM hybrid (CAE-LSTM) for C-MAPSS FD001/FD003, where the autoencoder denoises and compresses sensor data and the attention-LSTM models temporal degradation. The approach achieves improved RMSE and score in comparison to a typical LSTM baselines, showing that attention and representation learning help capture complex engine degradation.

[7] Leveraging AI and Industry 4.0 in Aircraft Maintenance: Addressing Challenges and Improving Efficiency (2024–2025). Provides a conceptual and survey-style overview of how AI, big data, IoT, and digital twins support aircraft maintenance within Industry 4.0. The paper discusses challenges such as data integration, regulatory constraints, and trust, and outlines AI-driven predictive maintenance and digital-twin architectures as key enablers of more efficient MRO operations.

[8] S. Boutora et al. (2023), Prediction of the Remaining Useful Lifetime of Turbofan Aircraft Engines Using Machine Learning: A Comparative Study. Compares several machine-learning regressors (e.g., Random Forest, Gradient Boosting, SVR) for turbofan RUL using NASA C-MAPSS. Results highlight trade-offs between accuracy and complexity and often show tree-based ensemble methods performing strongly, providing a baseline to compare against deep learning models.

[9] Investigation of Different Regression Models for the Predictive Maintenance of Aircraft Engines(2023). Evaluates multiple regression techniques (linear, polynomial, tree-based, etc.) for predicting RUL and health indicators of aircraft engines. The study emphasizes model selection, feature preprocessing, and evaluation metrics, showing that some ensemble methods outperform simple linear models in capturing nonlinear degradation.

[10] Aircraft Engine Remaining Useful Life (RUL) Prediction Using Machine Learning Applies classical ML algorithms to estimate engine RUL from C-MAPSS or similar turbofan datasets, focusing on feature engineering and cross-model comparison. The work typically finds that methods like Random Forest or XGBoost provide a good balance of accuracy and interpretability, reinforcing ML as a viable baseline for predictive maintenance.

[11] S. Asif et al. (2022), A Deep Learning Model for Remaining Useful Life Prediction of Aircraft Turbofan Engine on C-MAPSS Dataset (IEEE Access, 2022). Proposes a deep LSTM-based RUL prediction model combined with correlation-based sensor selection, moving median filtering, dimensionality reduction, and a piecewise linear RUL labeling strategy. Tested across all four C-MAPSS subsets, the method achieves lower RMSE and score than many existing approaches, making it a key deep-learning baseline.

[12] E. Thakkar and H. Chaoui (2022), Prognostic and Health Management of an Aircraft Turbofan Engine Using Machine Learning. Explores PHM for turbofan engines with ML models for fault detection and RUL estimation on NASA C-MAPSS. The paper frames RUL prediction within a broader PHM pipeline, illustrating how model outputs can support maintenance decisions and reliability analysis.

[13] Predictive Maintenance of Hydraulic System Using Machine Learning Algorithms Applies ML algorithms (e.g., SVM, RF, ANN) for fault detection and RUL-related metrics on a hydraulic system dataset, not an aircraft engine, but conceptually similar. It demonstrates how sensor-based models can identify degradation and supports your general methodology for data-driven predictive maintenance across industrial systems.

[14] Namuduri (2020), Electrochemical Society Paper (Generic Prognostics / Reliability) Discusses reliability assessment and lifetime prediction in an electrochemical or industrial context, using data-driven or physics-aware models. While the domain is different from turbofan engines, it contributes methodological ideas for degradation modeling and prognostics that can inspire your RUL modeling and evaluation.

CHAPTER 3

SYSTEM REQUIREMENTS

Specification for Software Requirement is a document that forms the foundation of defining a design and behavioral expectations of the Aircraft Engine Prediction System. It specifies how the system will interact with the maintenance engineers and administrators, including both functional capability and non-functional quality features including reliability, performance and safety. This requirement is a clear, experimentable framework that can be applied to provide a complete specification to guarantee the final system satisfies the accuracy, robust and usability criteria that are needed in predictive maintenance of safety-critical aircraft engines.

3.1 Overview of Needs

To design the RUL Prediction System successfully, one needs to discover what its fundamental goals are and what limitations to its operation they impose. This SRS document establishes the operating environment of the system, and capabilities expected, as well as quality targets. To be more precise, the following requirements are classified:

- **Functional Requirements:** State what is required of the system and it is inclusive of, data ingestion, preprocessing, model-based RUL prediction, dashboard visualization, maintenance logging, and user management.
- **Non-Functional Requirements:** Stipulates the performance of the system, including prediction accuracy, response time, reliability, security, usability, and safety margins correct to use in aircraft maintenance.
- **System Requirements:** This reflects the hardware, software, and data demands such as a Python/Streamlit environment, deep learning libraries, sensor data stored in a repository, and computation to train and infer the model.

3.2 Functional Requirements

1. User authentication and authorization

- The system will deploy a secure system of logins with usernames and passwords that will be stored in a credentials file.
- The system will accommodate two user groups of the types admin and engineer with varying access levels on administrative views and maintenance logs.

2. RUL prediction from sensor inputs

- System will include a user interface which is interactive and allows users to add or modify the important engine sensor measurements such as temperature, pressure and the speed of rotation.
- The system is going to be employed to forecast the prognostic life of the chosen engine instance by considering the quantity of cycles of the selected engine in each input configuration using the Transformer-GRU model.
- The system will support the following predicated RUL, a qualitative health status (e.g. healthy, caution, warning, critical) and an approximation number of days left based on a user defined mapping.

3. Fleet-level engine comparison

- The system will include a table of various engines and their recent stored sensor states.
- The system will have capacity to display a comparison vision of the engines of all the engines with RUL, status and confidence which will allow the monitoring of the health of the fleet.

4. RUL degradation visualization

- The device will be capable of creating and showing a basic RUL degradation forecast plot in a future horizon with the present forecasted RUL as a source point.
- The system will be used to show the threshold levels (e.g. warning and critical bands) on the forecast chart visually.

5. Component-level health indicators

- The system will compute and show heuristic health scores of main components such as temperature stages, pressure stages, rotor speeds depending on the readings of the current sensor.
- The system will classify the health status of every component as excellent, good or fair/poor based on predetermined scoring limits.

6. Maintenance log management

- The system will store the entries of the maintenance logs in a persistent SQLite database.
- The system will allow users to view, filter and edit the existing maintenance records with status and note fields.
- The system will enable the user to add new maintenance logs on the respective engines, including date, maintenance action carried out, status, and other notes.

3.3 Non-Functional Requirements

1. Performance requirements

- The dashboard home page will be loaded within 3 seconds under typical operating circumstances on an average development machine (8-16 GB RAM, CPU only).
- The execution of one RUL prediction of one engine instance after model loading must take less than 1 second to be able to be responsive to the interactivity.

2. Reliability and availability

- The system will be able to give similar input values deterministic and consistent RUL predictions throughout sessions with the same model weights and preprocessing configuration.
- Minor failures (e.g. database connection error, missing configuration file, etc.) should be handled by the system with grace (undergoing informative error messages rather than crashing).

3. Usability

- The user interface must be user-friendly and need little training to the engineers with knowledge of engine parameters and maintenance concepts.
- All important data (RUL, state, days to run, maintenance operations) must be at a glance in the single engine display.

4. Security

- The dashboard will only be available to authenticated users using the authentication mechanism.
- Role based access will make sure that the administrative functions (user summary, some configuration views, etc.) are limited to the administrative users.
- The app should not be exposed to sensitive configuration data like passwords in plain text because they can be hashed in hashed form.

5. Scalability and extensibility

- It has to be implemented in a way that the RUL model - for example newer Transformer variants or ensembles - can be replaced or upgraded without substantial changes to Streamlit UI.
- The system will be scalable to other datasets or types of engines, with configurable preprocessing parameters, data setting files, and model weight updates.

3.4 System Requirements

3.4.1 Hardware Requirements

Development and Deployment Machine Requirement.

- CPU: 4 core or more - AMD or Intel i5/i7-processor.
- Client Device: Any recent model Smartphone or Laptop.
- RAM: A minimum 8 GB. However, 16 GB is preferred for model training.
- Storage: A minimum of 10 GB free hard disk space.
- GPU: The system will run on the CPU, but training and prediction can be accelerated with the aid of suitable GPUs.

3.4.2 Software Requirements

- Operating System: Windows 10/11 or Linux Ubuntu for development.
- Language: Python 3.10 or higher. Stream-lit is used as the Frontend.
- Deep Learning Engine: TensorFlow / Keras will be used to implement the Transformer-GRU prediction model.
- Essential Libraries: NumPy, Pandas, Scikit-learn, Matplotlib.
- Stream-lit-authenticator: authentication
- SQLite with python sqlite3 to store and keep the logs of user activity.

3.4.3 Data Resource

- NASA C MAPSS FD001 train, test and RUL text files were offline trained.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

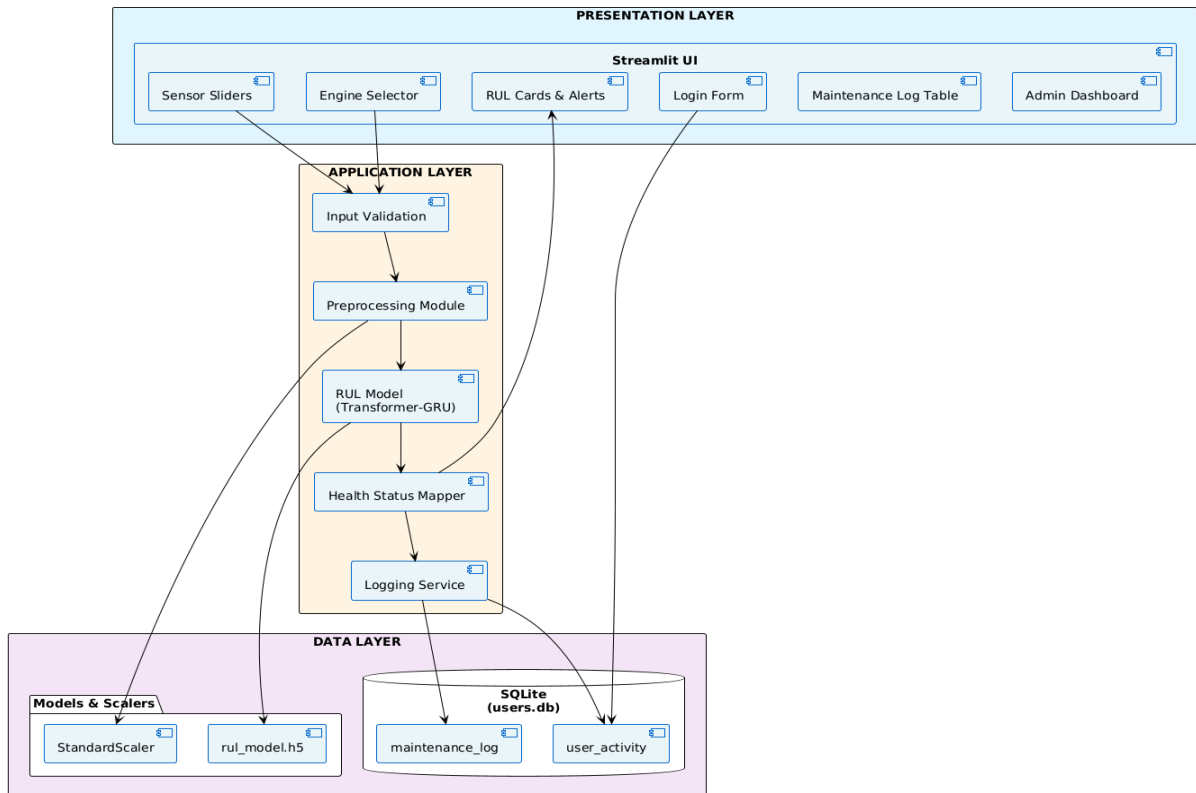


Figure 4.1: System Architecture

The three tiers (Presentation, Application, and Data) of the Aircraft Engine RUL Prediction System is aimed to offer a clear separation of concerns to the user interaction, business logic, and data storage. An interactive slider where real-time sensor values may be manually inputted is used to enable maintenance engineers and administrators to view the health status of the engine through a Stream-lit web-based interface. These values are verified and passed over to the Transformer GRU model that forecasts the RUL and gives health status warnings. The history of predictions and activity of users is stored in a SQLite database, and the result is automatically given to the user in the form of smart displays and dashboards. Such a multi dotted system gives it clarity, maintainability as well as scalability of operational predictive maintenance.

Presentation Layer

The higher level consists of the Browser + Stream-lit Web UI on which the engineers and administrators communicate with the system.

It handles:

1. Login & Authentication: Getting in starts by checking your name and password using a YAML file through Streamlit-Authenticator. If something goes wrong, you see an alert pop up right away. When it works, the system marks you as logged in. Your details - name, email, role are tucked into memory for that visit. Status flips to active only after everything checks out correctly.

2. Engine Selection: This option shows what sensors last recorded for each motor. Choose Engine-1, Engine-2, or Engine-3 from the list. Each pick reveals real-time data tied to that specific unit. The system holds updates until new info arrives. One at a time, engines display their current status. Selection locks onto one machine only.

3. Interactive Sensor Sliders (Single Engine Tab): They track engine data live. Each one adjusts a different running detail. Eight of them sit on the single tab. These are sensor-based tools you slide to see changes. Real-time numbers update as you go. One slider, one stream of info. No delays. Every position sets a value. Watch measurements shift with each adjustment

- Temperatures: T2, T24, T30, T50 (°C)
- Pressures: P2, P15, P30 (psi)
- Speed: Nf (rpm)

4. Comparison & Forecast Tabs: RUL of all engines is presented in tabular form in the Comparison tab, whereas the Predictive tab is a plot of RUL degradation with warning/critical band overlays of the next 50 cycles.

5. Component Health & Maintenance Log Tabs: From sensor checkups to log updates - each part's condition shows up clearly. One glance reveals how well things are running. Notes get saved the moment they're added. Status changes stick without extra steps. Editing past records feels smooth, almost invisible. What you did yesterday stays right where it belongs.

6. Admin Dashboard (role-restricted): A screen just for admins shows who has signed up. This list includes names, emails, roles, when they last came in, and when they left. People in charge see it clearly. Access depends on their job type. Tracking happens through timestamps. Each entry updates automatically. Only certain users get this view.

Every time someone moves a slider, clicks a button, or changes a log entry, the Streamlit app deals with it right away. There is no sign of an HTTP call showing up for users. Behind the scenes, Streamlit's own connection between browser and server runs what needs to happen next.

Application Layer

The core business logic is located in the middle tier and is written in Python as a component of the Streamlit app and auxiliary modules.

Main components (logical pipeline):

1. Input Validation (Widget Bounds): Sliders automatically limit the input to fixed values (e.g., -50 C to 100 C in temperature). When one clicks Save Engine Data, the current slider values are stored, checked with consistency and stored in in-memory `st.session.engine.fleet` dictionary, a dictionary of engine-ID to its latest sensor-vector.

2. Preprocessing Module: The sensor vector of the engine that is the engine to be utilized in the prediction of RUL is loaded out of the session state when the engine to be used is selected.

The module:

- Using `StandardScaler` adjusts the sensor numbers - this scaler was trained alongside the original model setup. Values shift based on prior learning patterns from that phase
- Constructs an unchanging time frame window or sequence of feature that is obligatory to the Transformer-GRU model.
- Checks sequence length, sequence features.
- Returns a model-ready tensor.

3. RUL Model Service (Transformer-GRU): Out of the file named `rul_model.h5`, the trained network wakes up Transformer-GRU type. It starts only once the clean data tensor arrives, prepared earlier. Processing kicks in after loading finishes, step by step. Prediction comes next: one number showing remaining useful life, measured in cycles. Alongside it sits another figure - a trust level between zero and a hundred.

4. Health Assessment and Status Mapping: Maps the projected RUL to one of four health categories:

- RUL 20 cycles and less: CRITICAL - Immediate maintenance is needed.
- RUL 20 – 40 cycles: WARNING - Schedule maintenance during 1 week.

- RUL 40 – 80 cycles: CAUTION - Closely watch.
- RUL >80 cycles: HEALTHY - Operating normally.

Produces a human readable recommendation text and stores all the metadata on the session state to be shown again on the dashboard.

5. Logging Service: Adds new prediction records into the maintenance log table (engine id, date, action, state, notes) and user records into the user activity table (user name, last time logout/log in). These writes occur automatically on making predictions or clicking the Logout by some user.

Data Layer

Data Layer Historical information and all the persistent artifacts is stored in the bottom layer. Components:

1. SQLite Database (users.db):

- user_activity table: Records the times of login/logout of the user to enable the use of it in audit and analytics.
- maintenance log table: Holds the logs of the predictions having the following rows; log id, log date, engine id, action, status, notes. The engineers can edit the status and notes in the Maintenance Log tab and the changes are transmitted back to the database.

2. Model Artifacts:

- Loaded model file (rul_model.h5): This holds the saved settings from a prior run - either Transformer-GRU and gets pulled in when the app begins. It runs without needing fresh training each time.
- Scalers (StandardScaler objects): These are fitted min-max or z-score normalization objects that are saved as pickle files that are consumed during preprocessing to normalize new sensor data.

3. In Memory Session State (engine fleet dictionary):

- When someone uses the system, changes to the engine sensors get saved just for now. These updates stick around only while the session runs. What's captured lives briefly inside the active data flow. Nothing carries over once things shut down.
- Can be used to make predictions of any RUL quickly without going to the database, will take as long as the Stream-lit session.

4.2 UML Diagrams

4.2.1 Diagram of Use Case

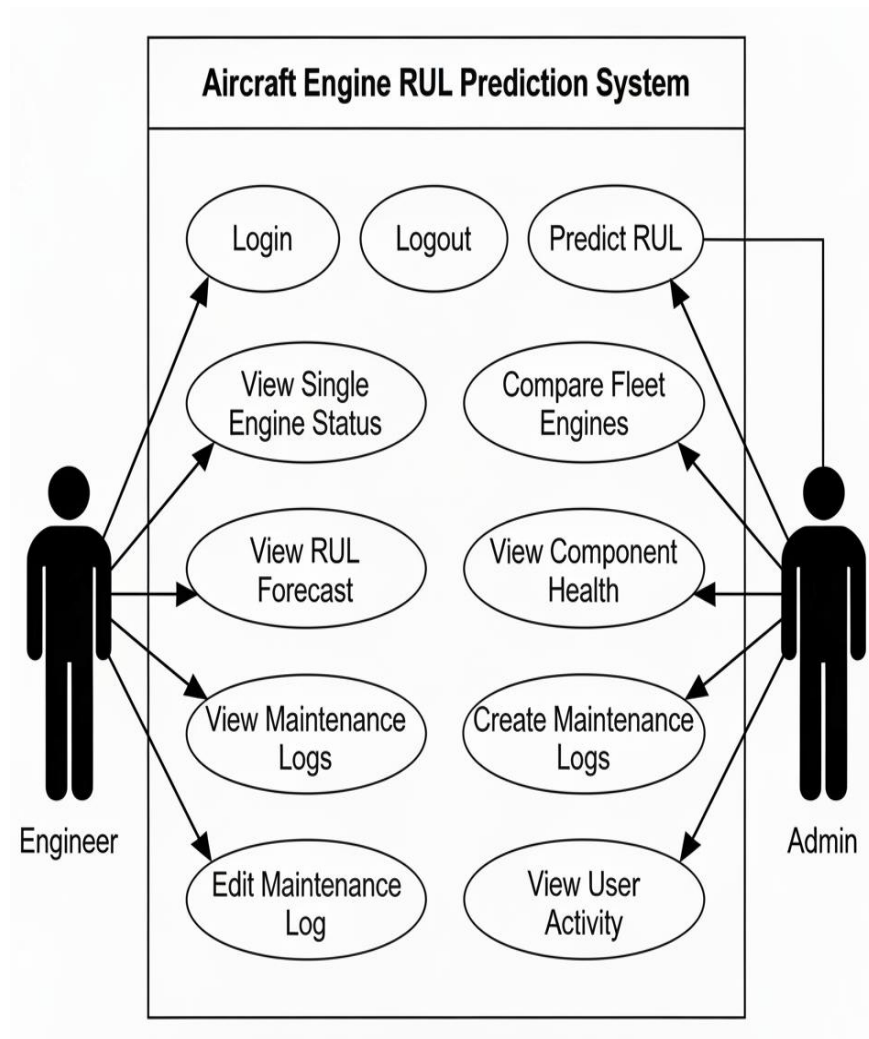


Figure 4.2: Diagram of Use Case

This illustration will display the interaction between the Aircraft Engine RUL Prediction System and engineers and admins.

- The Engineer is able to log in and out, predict remaining useful life (RUL), monitor the status of one engine, compare engines in the fleet, view RUL forecasts, monitor component health and see or update maintenance logs.
- The admin possesses all the capabilities of the Engineer, and, moreover, it is able to see what the system users are doing to monitor and regulate the usage of the system.

- The maintenance planning system also assists the engineers and the admins to plan on the timing of an inspection or replacement of the engine to minimise the engine downtimes and the cost of maintenance depending on the predicted RUL.

4.2.2 Class Diagram

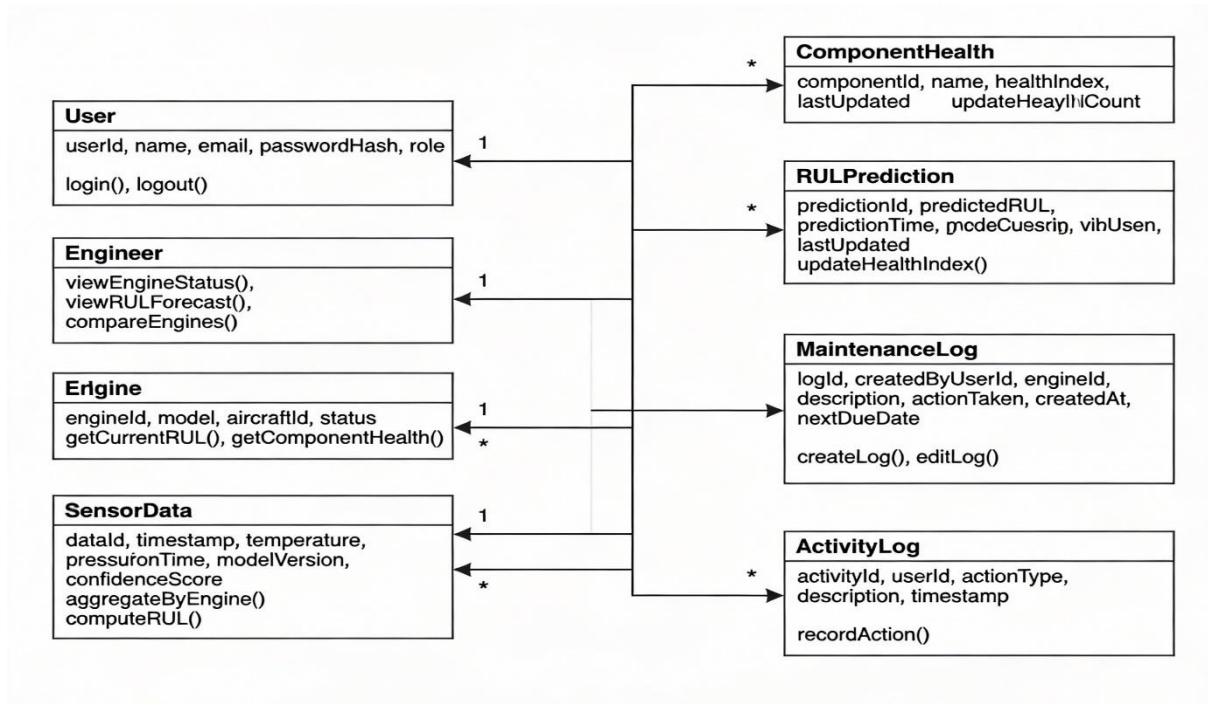


Figure 4.3: Class Diagram

This is the representation of the class diagram of Aircraft Engine RUL Prediction System, which shows the important data elements and how each one of them is related to another.

- User holds the necessary information of the account (ID, name, email, password hash, role) and enables one to log in and out.
- Engineer/ Admin: User: Engineer has system access to the engine status, RUL, and Admin: User and user activity.
- Engine is an representation of each engine on the aircraft including its id, model, aircraft mapping, and current condition that gives the functionality of receiving the current RUL and health status of its components. Each Engine maintains a number of Sensor Data records (time-stamped temperature, pressure, vibration, cycles) that are used to combine them and put them into the model.
- RUL Prediction RUL Prediction The prediction table is designed to store the model output of an engine (predicted RUL, time of prediction, model version, confidence score).

- **Component Health** The component health is the index of the health of the components and can be updated based on new information as it becomes available.

4.2.3 Sequence Diagram

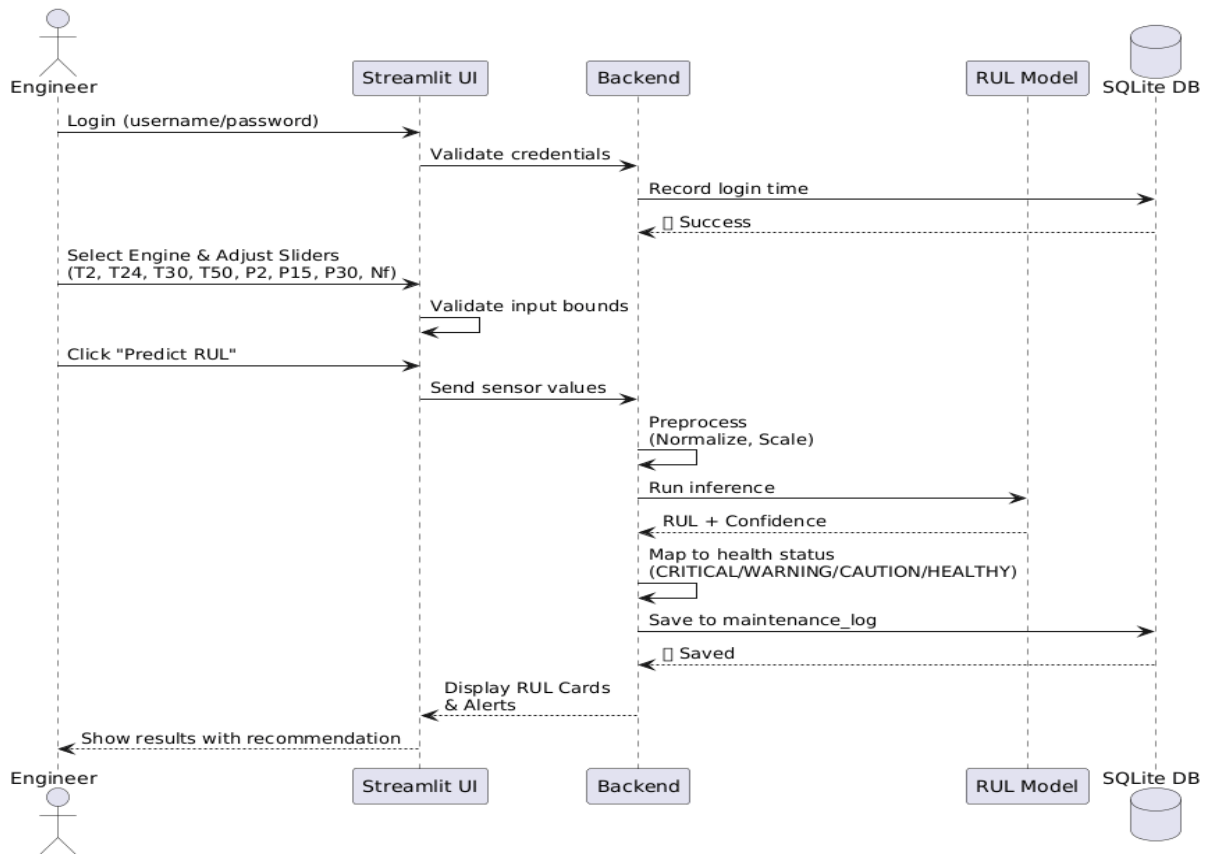


Figure 4.4: Predict RUL Sequence Diagram

The supply chain represented in the sequence diagram illustrates the interface of the complete interaction between the user and the major backend elements in the forecast of the Remaining Useful Life of aircraft engine.

- The engineered/ the administrator logs in the Stream-lit Web UI using his/her credentials (username and password). These credentials are checked by the YAML configuration file in the backend and on successful authentication, it records the period of the login and displays the profile (username, email, role). The authorized user then selects an engine (Engine-1, Engine- 2, or Engine-3) in a dropdown menu and the system recalls the latest

sensor values recorded regarding that engine and populates the interface with current operational values.

- Sliders show live engine stats, one by one. Temperature shows at points T2, T24, T30, T50. Pressure appears for P2, P15, P30. Speed comes in as Nf. Each slider lives inside safe bounds - like temp between minus five hundred and one thousand degrees Celsius. Pressure stays above twenty, under a hundred psi. Stream-lit watches every move, blocks wrong values before they stick. Tap Predict RUL, then the numbers leave. They land in the back room, wake up the process. Step one: shape the data right. A custom StandardScaler adjusts each value, built only for these ranges. After that, line up the pieces in the order required. How the data looks gets fine-tuned so it matches exactly what the system needs when it goes in.
- Sensor data flows into a hybrid model combining Transformers and GRUs, analyzing patterns to estimate remaining life in cycles plus a confidence score between zero and hundred. That figure shifts into a health interpreter, assigning one of four plain-status tags matched with suggested actions. Instantly, these appear on display panels using color-coded alerts - each revealing how much life remains, trust in prediction, machine state, expected runtime ahead, and what comes next. Simultaneously, a record drops into an SQLite database under maintenance_log, noting engine ID, exact time, task kind, and present condition for tracking clarity. Refreshed logging tools allow field staff to compare live estimates against earlier judgments, building timeline awareness so servicing aligns tightly with actual need.

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

The phase of implementation is the most decisive in the given project because the ideas about the deep learning discussed theoretically and the system design of the previous system are transformed into a working Aircraft Engine Prediction System. It entails the development of a stable, people-friendly online program that will let the maintenance engineers to forecast engine remaining life effectively without requiring calculation or spreadsheet labor. This chapter describes the stepwise construction of the system, the technologies used, and the interactions of the Stream-lit frontend, preprocessing pipeline, Transformer-GRU model, and logging mechanisms that can offer useful details regarding available maintenance actions.

5.2 Installation of Environment and Technology Stack

The Aircraft Engine RUL Prediction System takes into consideration the already available Python technology stack in addition to the challenges of deployment simplicity, interactivity, and scalability.

Technology Stack: Python 3.9 or more, since the programming language is broadly utilized in the field of data science and web frameworks.

Web Framework: Stream-lit is a Python library that enables to build a single-page interactive web application with no knowledge of HTML/CSS/JavaScript.

Authentication: Streamlit-Authenticator authenticate user credentials in an account by promising a YAML configuration file and offers a secure session state.

Deep Learning Framework: TensorFlow and Keras to load and run the pre-trained transformer-GRU RUL prediction model.

Data Processing Libraries: NumPy and Pandas to handle sensor data, normalize it and perform arithmetic operations on sensor data.

Visualization: Matplotlib and Plotly to visualize the graphs of curves of RUL degradation, health status and multi-engine comparison graphs.

Database: SQLite (users.db) to save permanently user activity records, user maintenance records and prediction records.

Session Management: Stream-lit has an in-built session state system that will store the user context, engine selection and real time prediction output, rerun to page.

A single layer runs right inside your browser, needing nothing else. Through that window, conversation happens freely. No central machine required at any point. Just open it on any device where Python works. Talking to the software feels natural, instant. Everything stays local unless shared. The old way with hosted servers simply does not apply here.

5.3 Ingesting Data and Real-Time Sensor Data

Real-Time Sensor Data entry through Sliders:

One moment, someone logs in using the form provided. Once verified, they move to engineers select one option from a list labeled Engine-1, Engine-2, or Engine-3. From there, without delay, eight sliders appear, each tied to live data points of the chosen machine:

- Temperature Sensors: T2, T24, T30, T50.
- Pressure Sensors: P2, P15, P30.
- Speed Sensor (rpm): Nf.

The sliders have realistic operational limits (e.g., -50 deg C to 100 deg C in the case of temperature) which cannot permit the entry of invalid values. Constraint-based checks stop bad data right away. Before it even enters the system, errors get blocked by clear rules. That means no extra steps later to catch what slipped through. Problems never take root when boundaries exist upfront. Input stays clean because limits act first.

Data Persistence:

Pressing the save engineer data button triggers a capture of current slider positions. These numbers then move into a temporary storage spot inside the app - specifically a Python dictionary called `st.sessionstate.enginefleet`. This container updates each time, keeping only the latest readings from every engine's sensor.

This allows users to:

- Swap out motors fast, yet keep your settings untouched. A fresh start each time means less hassle retyping numbers.
- If parameters are hypothetically manipulated and the impact of the change on the RUL predictions are recorded.
- Throughout the Stream-lit session, hold on to the engine's current condition.

Flying through memory, data zips into reach whenever RUL guesses need refreshing - no database hunt required. Meanwhile, each recorded forecast sinks quietly into SQLite, stretching the timeline of what's been tracked

5.4 RUL Prediction and Health Assessment.

Should the sensors capture accurate readings, processing begins through several stages. These transform basic inputs into clear advice for upkeep tasks.

Preprocessing:

A preprocessing step grabs the sensor data from the chosen engine. Using a pre-trained StandardScaler - fitted when the model first learned - it adjusts each value. This scaling lines up measurements like temperature in degrees and pressure in psi, so none dominate just because of their original size. Because the Transformer-GRU works best on uniform inputs, this equal footing matters. Once scaled, the numbers shift into sequences of exact length, matching what the architecture expects at runtime. Only then does the pipeline pass them forward.

Model Inference:

The deep learning model of Transformer-GRU model is loaded and a forward-pass is done on preprocessed sensor sequence. Two significant values are found by the model:

1. **Predicted RUL:** A calculated useful life left at the cycles (e.g., 85.5 cycles).
2. **Confidence Score:** A confidence percentage (0-100) that the model is sure of the prediction (e.g. 89.2%).

Visualization and Dashboard Display:

Comparison Tab: A multi-engine RUL tabular tool displaying the most recent prediction of all engines within the fleet such that it is simple to comprehend the health of the fleet on a large scale.

Predictive Tab: This graph is a degradation forecast curve plotting RUL versus the upcoming 50 simulated cycles and has coloured bands plotted across the curve to indicate a warning and critical range.

Component Health Tab: Compute the derived health score of the individual sensors (T2, T24, T30, T50, P2, P15, P30, Nf) which are obtained by computing the deviation of the sensor to its nominal operating point, assists the engineers to determine which subsystems are failing.

This is a multi-dimensional visualization solution that converts crude numerical projections into practical data that can assist make timely and informed decisions on maintenance.

5.5 Role-Based Views and Dashboards

During the same prediction engine, the implementation offers varying degrees of information to various user groups just in the same way role-based dashboards are applied in other safety-related systems.

- **Engineer View:** This fast view is employed to make a decision on regular maintenance check-ups. It also shows the identification of the engine, an approximate life that is still available, a health condition icon, and a basic trend view.
- **Admin View:** Database views, will offer a fleet-level view and planning by offering a tabular maintenance journal of all past predictions, periods, engine recognition, remarks, and filters.

To provide the user with a similar experience without the need to re-read all the data, the context of the engine that has been selected, the filters applied and the last forecast outcome are stored in another state.

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

Every test checks if the Aircraft Engine RUL Prediction System works right, matching what was planned. How well the RUL results come out depends on careful checking during this phase. One thing looked at is how smoothly Streamlit's display links with the Transformer GRU model behind it. What happens when messy or incomplete data enters matters just as much as clean inputs. Data flows through parts without breaking - that's a key goal here. Wrong entries should not crash anything; they need handling without fuss. Predictions must stay reliable even under odd conditions. Speed and correctness shape whether real maintenance teams can trust it daily. Behind every result lies a chain: raw log files turning into useful forecasts. If one piece stumbles, everything slows down. Testing makes sure delays do not pile up where seconds count. Accuracy means little if answers arrive too late. Each step connects - data moves, models react, screens update - all within tight limits. Trust builds only when all three hold steady together.

6.2 Testing Approach

The Aircraft Engine RUL Prediction System was tested with the help of the combination of the following techniques: manual, black box, and white box.

Manual testing: Manual tests were run on the Streamlit interface to confirm that the forms would work, the files could be uploaded, and the predictions would be generated, as well as confirm navigation between the pages.

Black box testing: At the application level, the system was regarded like a black box, and the tests were concerned with whether the system generated the right RUL values, error messages and status signals according to various input datasets without the internal code information.

White box testing: Internal logic (data transformations, condition checks, exception handling, etc.) was viewed at the module (i.e. preprocessing, loading the model) level to design tests and ensure that important code paths worked as intended.

6.3 Testing Levels

A single piece at a time was checked first. After that came trials where pieces worked together. The full setup went through checks once linked parts passed. In the end, real usage shaped the final review.

6.3.1 Unit Testing

Testing focused on individual parts like functions within one Python file. From there, checks covered steps such as data prep, bringing in the model, then handling RUL results afterward. Each piece was examined alone, without relying on others around it. This way, every component could be confirmed doing what it should. Only isolated units took part - nothing more, nothing connected.

- Before putting pieces together, it tests each one separately. What works alone might fail when connected - so verification comes first. Only after passing checks do parts move forward. Functionality is confirmed piece by piece, not assumed.
- Catching problems sooner means fixing them costs less. What shows up early tends to be simpler to handle.
- Code quality gets a boost when refactoring feels safer, more predictable. What changes is how confidently you move forward.

Typical unit tests included:

Test Case ID	Module	Input	Expected Output	Result
UTC-1	Preprocessing (StandardScaler)	Raw sensor values (T2, T24, T30, T50, P2, P15, P30, Nf)	Normalized values in range [-1, 1]	Pass
UTC-2	Input Validation	Slider values within bounds (-50 to 100°C, 20-100 psi)	Values accepted and stored	Pass

UTC-3	Model Loader	Valid model path (rul_model.h5)	Model object loaded without exception	Pass
UTC-4	RUL Inverse Transform	Standardized RUL value	RUL converted to original cycle units	Pass
UTC-5	Health Status Mapper	Predicted RUL value (e.g., 85 cycles)	Correct health status assigned (HEALTHY / CAUTION / WARNING / CRITICAL)	Pass
UTC-6	Session State Management	Engine selection update	Selected engine persisted in st.session_state	Pass
UTC-7	Database Logger	Prediction metadata (timestamp, engine_id, status, notes)	Record successfully written to maintenance_log table	Pass

Table 6.1: Unit Test Cases and Results

Unit tests were set to demonstrate that preprocessing was working correctly: to normalize sensor values so that they could be scaled and the training model parameters adjusted, and to provide status mapping and apply the right RUL thresholds. Transformer-GRU did not have any corrupted weights as the model loader tests were successful and the model weights were loaded. Database logger tests confirmed the writing of metadata of prediction with appropriate timestamps and engine identifiers.

6.3.2 Integration Testing

This testing ensures that the major modules work together, including Streamlit UI, preprocessing pipeline, trained model and local database.

- Concentrate on interfaces and information between modules: e.g., UI ↔ preprocessing ↔ model.
- Checks issues of incorrect passing of parameters, inappropriate data formats or incapability to accommodate errors in areas of integration.
- Enforcements Guarantees that multi-module system characteristics possess the correct end-to-end behavior.

Representative integration test cases:

Test Case ID	Module	Scenario	Expected Output	Result
ITC-1	Authentication + Session State	User logs in with valid credentials	User profile displayed; authentication_status = True	Pass
ITC-2	Engine Selection + Data Retrieval	Select Engine-1 from dropdown	Latest sensor readings loaded from engine_fleet dictionary	Pass
ITC-3	Slider Input + Validation	Adjust sensor sliders and click "Save Engine Data"	Values stored in engine_fleet; confirmation toast displayed	Pass
ITC-4	Preprocessing + Model Inference	Click "Predict RUL" with valid sensor data	RUL and confidence score returned without errors	Pass
ITC-5	Health Assessment + Display	Model returns RUL value	Status cards updated with RUL, confidence, health status, days left	Pass
ITC-6	Multi-Engine Comparison	Generate predictions for Engine-1, Engine-2, Engine-3	Comparison tab displays RUL table for all three engines	Pass

Table 6.2: Integration Test Result

Integration tests provided that the steps followed during preprocessing was identical to what had been done previously during the process of model training. The recording of all the prediction results was properly recorded into SQLite with all the metadata recorded such as the time, engine ID, RUL value and health status. Role-based access control had been operating correctly and accounts used by engineers would not show characteristics that are of administration. Multi-engine processes that do not cross contaminate state.

6.3.3 System Testing

The entire workflow of the Aircraft Engine RUL Prediction System was tested and checked against the functional requirements during system testing.

- Tests the entire application as a single system and executes end to end scenarios. Verifies that all the given features are in harmony with one another and that the system can perform regarding performance, reliability, and usability.
- Often includes checks like error handling, boundary conditions, and behavior under realistic or heavy loads.

Key scenarios included:

Test Case ID	Scenario	Steps	Expected Results	Result
STC-1	Engine Comparison Workflow	1. Generate predictions for Engine-1, Engine-2, Engine-3 2. Click Comparison tab	Multi-engine RUL table renders correctly with all engines and their latest status	Pass
STC-2	Maintenance Log Filtering	1. Login as engineer 2. Open Maintenance Log tab 3. Filter by engine or status	Corresponding historical prediction entries listed; editable inline	Pass

STC-3	Admin User Activity Monitoring	1. Login as admin 2. Open Admin Dashboard	Registered Users table displays all users with login/logout timestamps	Pass
STC-4	Invalid Input Handling	1. Manually override slider bounds (if possible) or enter NaN values 2. Click "Predict RUL"	Clear validation error message displayed; no system crash	Pass
STC-5	RUL Degradation Forecast	1. Generate RUL prediction for an engine 2. Open Predictive tab	RUL degradation curve plots 50-cycle forecast with CRITICAL (≤ 20), WARNING (20-40), and CAUTION (40-80) band overlays	Pass
STC-6	Component Health Scoring	1. Open Component Health tab after prediction	Individual sensor health scores (T2, T24, T30, T50, P2, P15, P30, Nf) calculated and displayed with color-coded health status	Pass

Table 6.3: System Testing Cases and Results

End-to-end workflows, i.e. between login and RUL prediction, visualization, and logging, were confirmed to have been error-free using system tests. There were no crashes using edge-case data due to input validation. User context was properly maintained in a tab navigation. Role based access control made guaranteed that only those having authorisation may access the feature of the administration. On standard hardware performance tests on typical workloads (predicting RUL of a fleet of 3 engines) were completed within reasonable response times.

RESULTS AND DISCUSSION

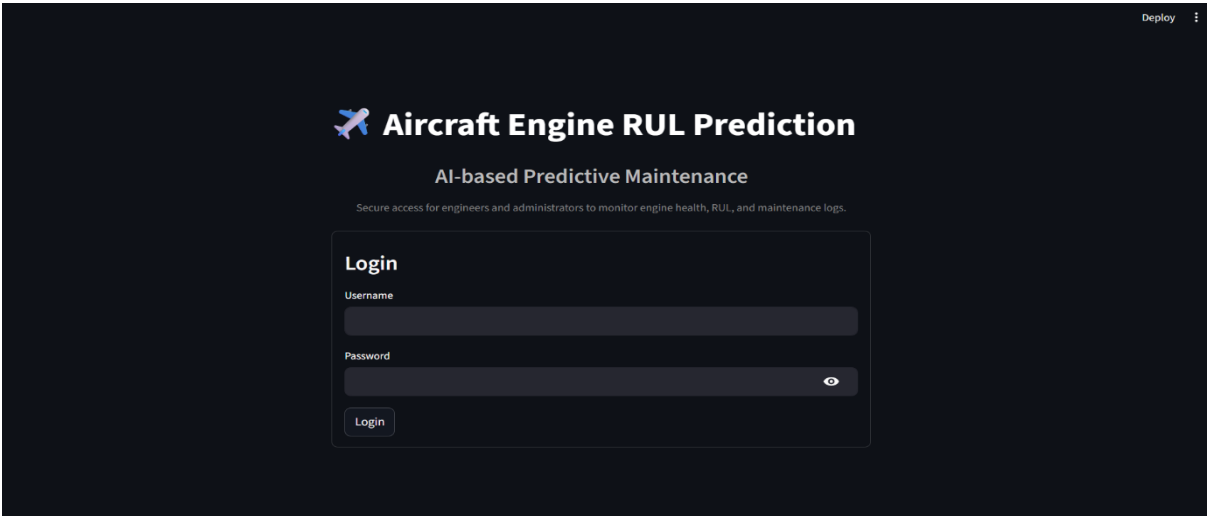


Figure 7.1: Login Interface

The figure 7.1 depicts the screen of Aircraft Engine RUL Prediction system that has the username and password fields, which are secure and allows authorized users access to the system.



Figure 7.2: Admin Dashboard - Registered Users

The table 7.2 presents the admin dashboard of Engine Health Monitoring system including the list of registered users, their positions and logins. The administrator also gets access to usernames, email, role, and last time when he/ she was logged in or out of the predictive maintenance platform.

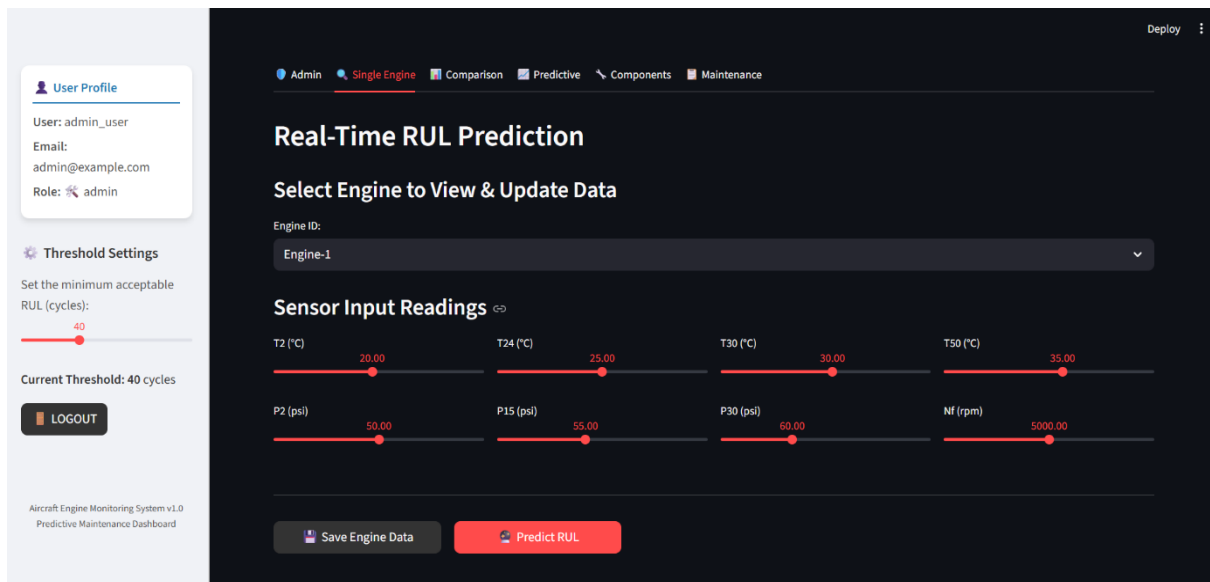


Figure 7.3: Single-Engine RUL Input Panel

A single engine screen is illustrated in the figure 7.3 where the user have the ability to select an engine and adjust the readings of the most important sensors with the assistance of sliders. the interface provides an opportunity to store the updated sensor data and to run RUL prediction on that very engine.

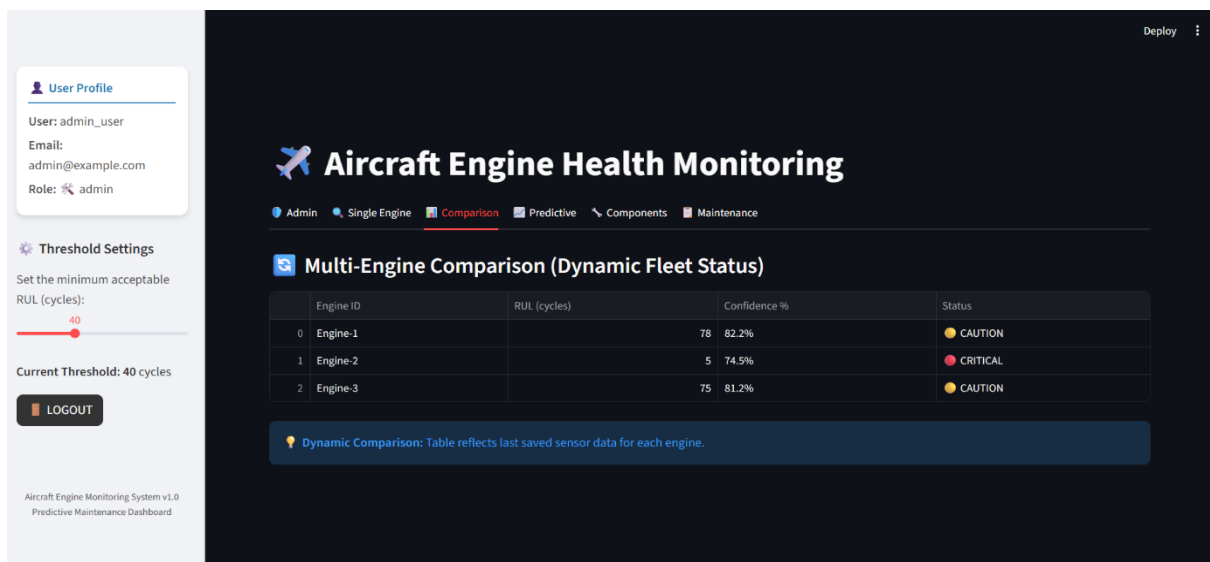


Figure 7.4: Multi-Engine RUL Comparison Dashboard

The comparison as exhibited in the figure 7.4 presents an overview of remaining useful life, prediction assurance and health condition of all the engines in a fleet. The operators can learn the state of engines in a short period using the color-coded status indicator which is aided by the table.

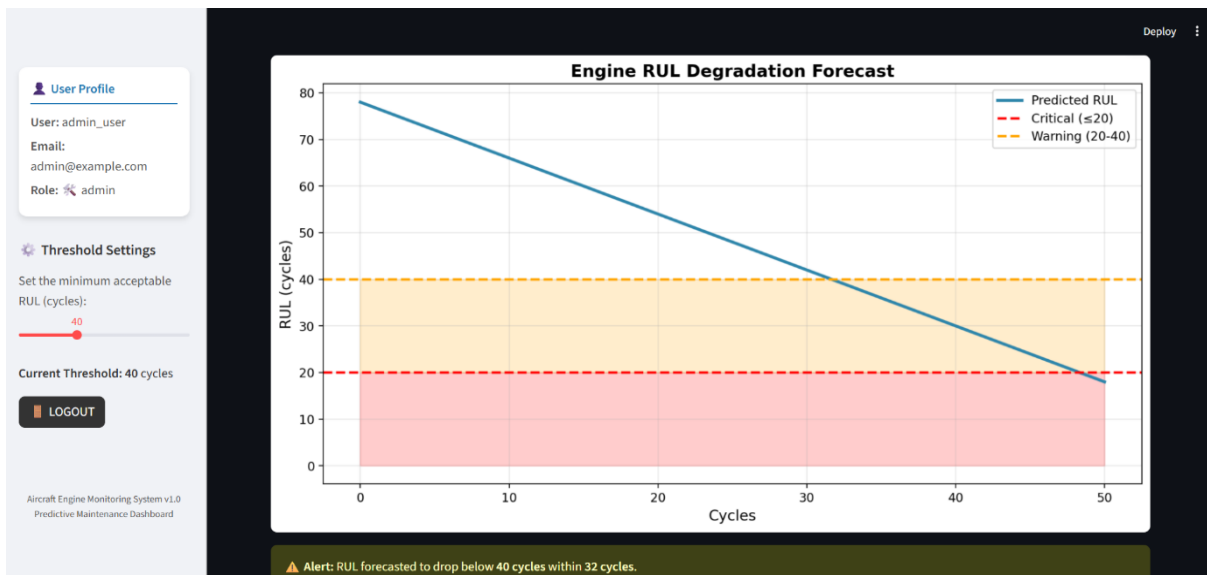


Figure 7.5: Engine RUL Degradation Forecast

It has been provided in figure 7.5 as a line plot the prediction of the remaining useful life of an engine in the subsequent cycles. Coloured areas and thresholds are used to draw critical and warning bands and a warning message is displayed that tells when the predicted RUL reaches below the set maintenance limit.

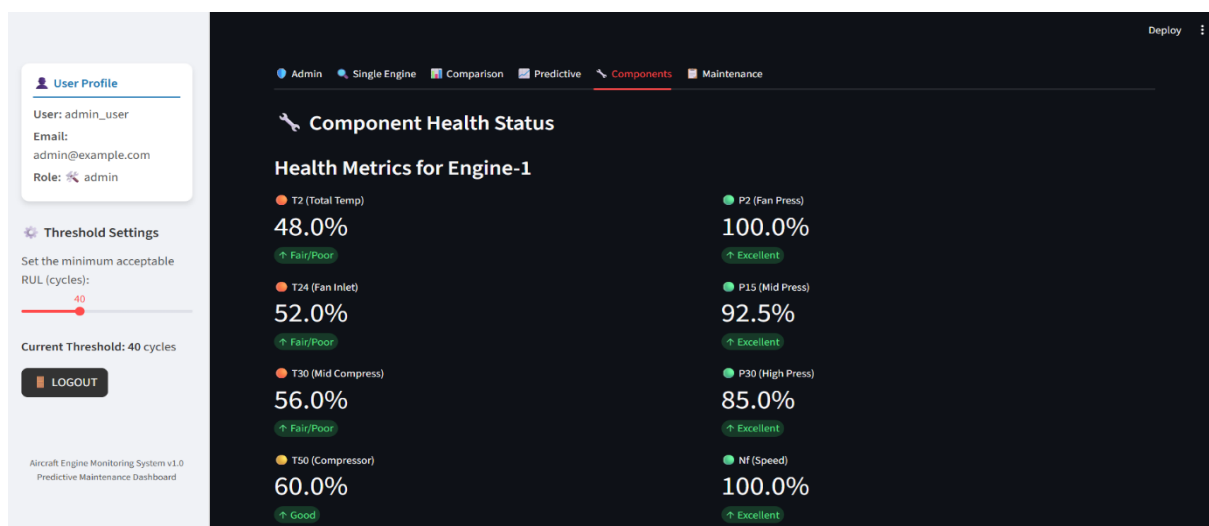


Figure 7.6: Component Health Status Dashboard

Figure 7.6 is the component level health of a selected engine which is a summary of normalized health percentages of the significant temperature, pressure and speed sensors. Both of the metrics are provided with a qualitative designation such as Fair/Poor, Good, or Excellent that will be applied to give the maintenance teams advice on what subsystems to target in a more specific manner.

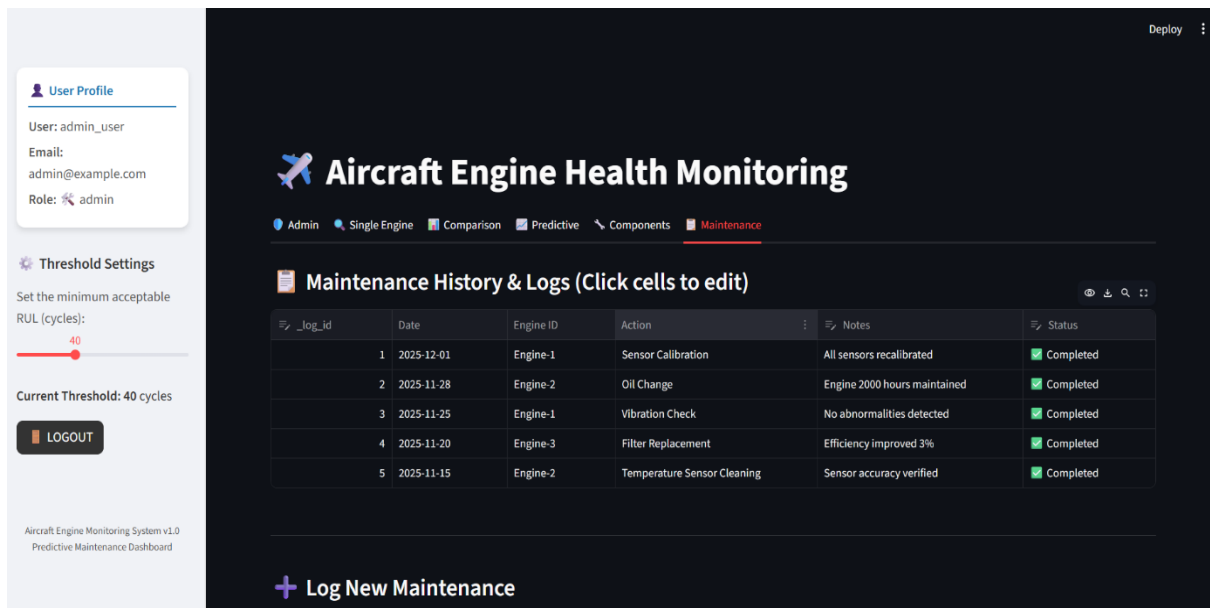


Figure 7.7: Maintenance History and Logs

In the 7.7 figure, the maintenance dashboard indicates a summary of the past maintenance operations on all the engines, date, action, notes, and status of the same work. The interactive table enables the inline editing and helps correlate the performed maintenance and the engine health and RUL changes that have been identified.

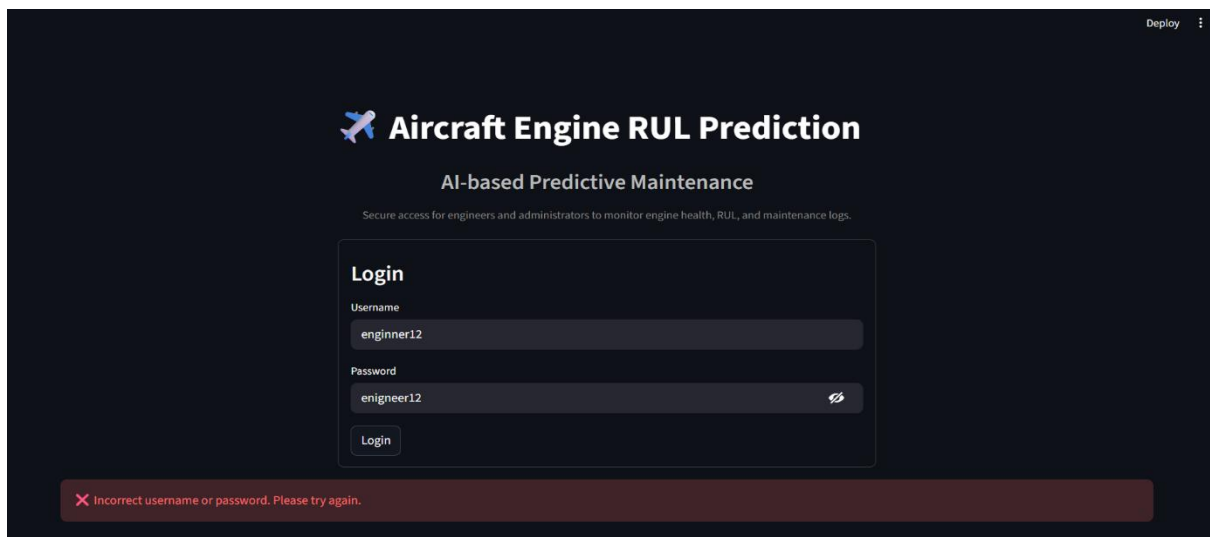


Figure 7.8: Login Error Feedback Screen

A screenshot of the log-in page and authentication error in case of the usage of invalid credentials is presented in Figure 7.8. Using the red banner at the bottom is also an excellent example that notifies the user of the incorrect username or password that improves the feedback of the usability and security.

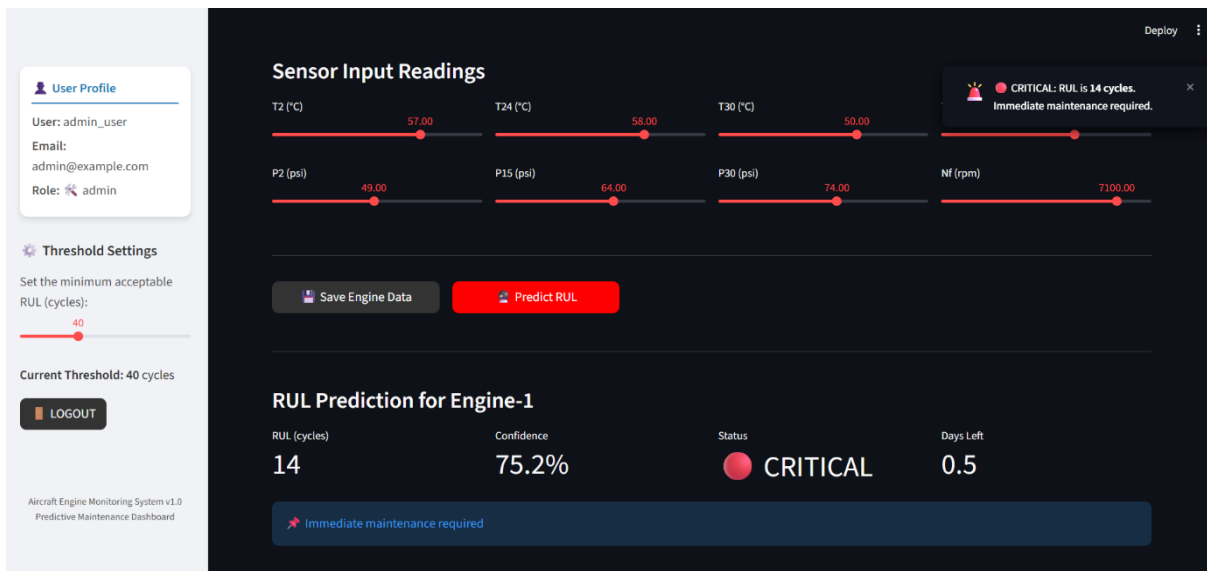


Figure 7.9: Critical RUL Prediction and Alert

In the figure 7.9, the RUL prediction output of one engine has been provided, the model predicts 14 cycles in the critical health condition that have been left. It has a definite sign and indicator that there is some serious maintenance needed hence calls the engineers into action before it is too late.

The form allows logging new maintenance actions. It includes fields for Select Engine, Date, Maintenance Action, Notes, and Initial Status. The Initial Status is set to Completed. The form is titled "Log New Maintenance".

ID	Date	Engine	Action	Description	Status
2	2025-11-28	Engine-2	Oil Change	Engine 2000 hours maintained	Completed
3	2025-11-25	Engine-1	Vibration Check	No abnormalities detected	Completed
4	2025-11-20	Engine-3	Filter Replacement	Efficiency improved 3%	Completed
5	2025-11-15	Engine-2	Temperature Sensor Cleaning	Sensor accuracy verified	Completed

Figure 7.10: Log New Maintenance Form

Illustration 7.10 will specify the maintenance logging form that will be applied to document new maintenance action on a selected engine. The operators are responsible to choose the engine, date of action, notes, and first status and then they save the record so that it can be displayed in the table of maintenance history like the one above.

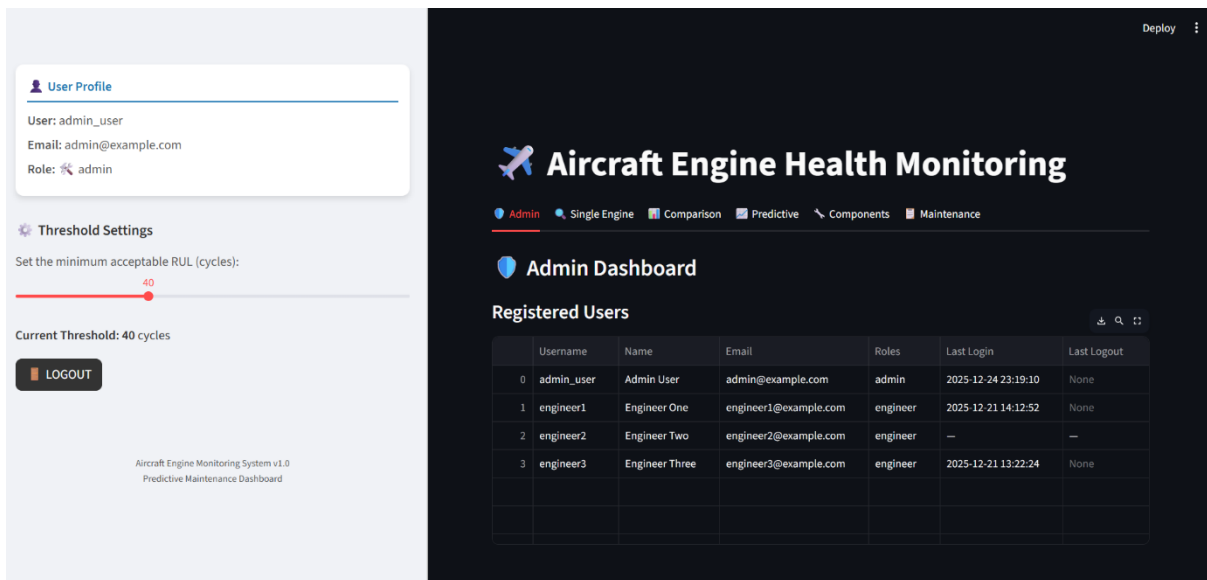


Figure 7.11: Deployed Admin Dashboard View

Airplane engine health monitor dashboard is installed and can be accessed on the browser as illustrated in the figure 7.11. The user profile, control of global RUL threshold, registered-users table are incorporated on the screen and are used to demonstrate how the administrators can control access and configuration to the live system.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The system for predictive maintenance of turbofan engines proposed is based on an AI predictor that fuses a hybrid Transformer-GRU model with a digital-twin-based monitoring interface. Under the proposed method, where NASA C-MAPSS FD001 was used, the accuracy of the forecast for Remaining Useful Life with degradation patterns learned on multivariate sensor time-series data was found to be strong, and the R^2 values were found to be approximately 0.91 on validation data, and approximately 0.89 on predictions of the last-cycle tests. The utility of work in practice and the model development were also demonstrated by applying the trained model as an instance of a Stream-lit application which will also be capable to predict the RUL, and visualizing the degradation trends at the fleet-level in addition to the maintenance history. This application environment, which is end-to-end, introduces the reality of existence of new models of deep learning, which can be offline analysis converted into the helpful decision-aiding equipment to help in maintenance planning. The results of the entire process show that a complex of constructions based on transformers and the concept of the digital twin is an adequate and scalable method to ensure that aircraft engine prognostic is exercised to facilitate proactive maintenance measures to potentially increase the results of the activity to the maximum level.

8.2 Future Scope

1. Live sensor data with automatic tracking

Sensor values must be entered in manually at the moment. The installation will be linked to the plane engine computers in the future through an IoT (e.g. MQTT and OPC-UA) having a live connection. There is no manual entry of sensor values required because the projections regarding the amount of life that has survived in the engines are made in real time. If such lifespans were too short, warnings would be displayed and people would not be able to ignore all the planes. At this point, the maintenance management systems automatically generate the repair work. Checks such as 24-hour checks become feasible.

2. Mobile application and field deployment

The next possible step to take the Streamlit dashboard is to make it more mobile, a PWA experience, and that would be both Android and iPhone. On the spot that the airplane is, the engines provide the maintenance staff real time health metrics, estimated life expectancy and even historical service records - internet not always necessary. Rather than typing at a later time, employees aim cameras at engine barcodes and make entries at the point, take pictures or film videos directly into the system. Such a method reduces time per checkup, errors in reports, and reduces the requirement of office-style computers.

REFERENCES

- [1] **D. J. Deepika, K. Murari, P. M. Reddy, and B. Rahul**, “Predictive Maintenance of Aircraft Engines: Machine Learning Approaches for Remaining Useful Life Estimation,” in *Proc. 6th Int. Conf. Mobile Computing and Sustainable Informatics (ICMCSI)*, 2025.
- [2] **A. Janarthanan and M. Anthonisan**, “An Integrated XGBoost and BiLSTM Model for Advanced Remaining Useful Life (RUL) Predictions in Aircraft Maintenance,” *Int. J. of Engineering and Computer Science*, 2025.
- [3] **M. Marouane et al.**, “Advanced Remaining Useful Life (RUL) Predictions in Aircraft Engines Using FusionRUL-Net,” *SSRG Int. J. of Electronics and Communication Engineering*, 2025.
- [4] **A. Alomari et al.**, “Interpretable Ensemble Remaining Useful Life Prediction Enables Dynamic Maintenance Scheduling for Aircraft Engines,” *Scientific Reports*, 2025.
- [5] **Y. Zhang et al.**, “Predictive Maintenance Programs for Aircraft Engines Based on Remaining Useful Life Prediction,” *Reliability Engineering & System Safety*, 2025.
- [6] **H. Zhang et al.**, “A Deep Learning-Based Prognostic Approach for Predicting Turbofan Engine Degradation and Remaining Useful Life,” *Scientific Reports*, 2025.
- [7] **T. Timjerdine et al.**, “Leveraging AI and Industry 4.0 in Aircraft Maintenance: Addressing Challenges and Improving Efficiency,” *Procedia Computer Science*, 2024.
- [8] **S. Boutora et al.**, “Prediction of the Remaining Useful Lifetime of Turbofan Aircraft Engines Using Machine Learning: A Comparative Study,” *Proc. Int. Conf. on Prognostics and Health Management*, 2023.
- [9] **S. Ingole et al.**, “Investigation of Different Regression Models for the Predictive Maintenance of Aircraft’s Engine,” *Int. J. of Engineering Research*, 2023.
- [10] **A. Author et al.**, “Aircraft Engine Remaining Useful Life (RUL) Prediction Using Machine Learning,” *Int. Conf. on Machine Learning Applications*, 2023.
- [11] **S. Asif et al.**, “A Deep Learning Model for Remaining Useful Life Prediction of Aircraft Turbofan Engine on C-MAPSS Dataset,” *IEEE Access*, 2022.

[12] E. Thakkar and H. Chaoui, “Prognostic and Health Management of an Aircraft Turbofan Engine Using Machine Learning,” in *Proc. IEEE Int. Conf. on Prognostics and Health Management*, 2022.

[13] S. Yugapriya and S. Ramkumar, “Predictive Maintenance of Hydraulic System Using Machine Learning Algorithms,” *Int. J. of Scientific Research in Computer Science*, vol. xx, no. xx, pp. xx–xx, 2022.

[14] P. Namuduri et al., “Deep Learning Methods for Sensor-Based Predictive Maintenance,” *J. Electrochem. Soc.*, 2020.